

# nanochat on a Laptop CPU: a Three-Minute End-to-End Example for the AI-Human Research OS

September 2026

## Abstract

We shrink OpenResearch’s default nanochat demo until every stage, from tokenizer training through pretraining, evaluation, supervised fine-tuning (SFT), and chat, runs on a laptop-class CPU. With a one-layer, 0.84M-parameter model, a 4,096-token vocabulary, and 300 pretraining steps, the full pipeline, including two probe runs, finishes in 2 min 57 s on an Intel i7-12700, using at most 1.0 GB of RAM per process. Validation bits per byte (bpb) falls from 3.456 to 2.288 during pretraining and from 2.289 to 2.146 during SFT. Doubling the Muon matrix learning rate lowers the final bpb by 0.012, and halving the vocabulary raises it by 0.096. The model produces no useful answers: the contribution is a runnable, deterministic first example for the Research OS, not a research result.

## 1 Introduction

The AI-Human Research OS organizes research as plain files that a human and code agents share. Its users need one example project they can run end to end on their own machine, to see the idea, code, figures, results, and paper live together. OpenResearch [2] ships such a demo: it trains nanochat [6], a compact pipeline for building a chat model, at depth 6 with 73.5M parameters for 5,000 pretraining and 1,500 SFT steps on an Apple M3 Max. Its pretraining alone would take over nine hours on the CPU used here.

This project ports that demo and scales it down until the whole pipeline takes about three minutes on a CPU. It contributes (i) a one-command, staged, CPU-only port whose code changes are documented; (ii) measurements of the time, memory, and learning curves of every stage; and (iii) the demo’s two probes, a larger matrix learning rate and a smaller vocabulary, run at this scale.

## 2 Setup

**Code.** The port starts from OpenResearch commit 27cb342, path `demo/nanochat`, with the demo’s overrides applied. The code changes are small: `torch.compile` becomes switchable and is off on CPU, because the Windows CPU backend needs a C++ compiler; SFT data can be downloaded as an evenly spaced subsample; and CORE prompts are cropped to the rotary cache length. A staged runner executes each stage in under a minute and logs its exit code, wall time, and peak memory.

| Knob                    | Demo                             | This example                               |
|-------------------------|----------------------------------|--------------------------------------------|
| Model                   | depth 6, width 384, 73.5M params | depth 1, width 64, 0.84M params            |
| Tokenizer               | 32,768 vocab, 2B chars           | 4,096 vocab, 50M chars                     |
| Context / batch         | 512 / 16,384 tokens              | 256 / 4,096 tokens                         |
| Pretraining data        | 8 shards ( $\sim 0.8$ GB)        | 1 train + 1 val shard ( $\sim 184$ MB)     |
| Pretraining steps       | 5,000 (81.9M tokens)             | 300 (1.23M tokens)                         |
| SFT data                | full splits ( $\sim 980$ MB)     | $\sim 1/128$ of each split ( $\sim 14$ MB) |
| SFT steps / LR fraction | 1,500 / 0.8                      | 40 / 0.1                                   |
| Base evaluation         | CORE + bpb + samples             | bpb + samples                              |

Table 1: Recipe of the OpenResearch demo and of this example.

**Recipe.** Table 1 compares the demo’s recipe with this example. The data-to-parameter ratio of pretraining stays close to the demo’s (3.95 versus 3.5 tokens per scaling parameter). Pretraining uses one training shard and the pinned validation shard of `climbmix-400b-shuffle` [7]. SFT uses the demo’s mixture of SmolTalk [1], MMLU [4], and GSM8K [3], subsampled to about  $1/128$  of every split. Matrix parameters are trained with Muon [5], as in nanochat. SFT starts at 0.1 of the inherited learning rate: at the demo’s 0.8, SFT validation bpb spiked from 2.29 to 2.63 and ended above its start.

**Hardware and metric.** All runs use Windows 11 on an Intel i7-12700 (12 cores, 20 threads) with 32 GB of RAM and no GPU, in eager mode. The metric is validation bits per byte (bpb), measured every 25 pretraining steps on 65,536 validation tokens and every 10 SFT steps on 32,768 tokens. Training is deterministic on this machine: two fresh runs gave identical validation bpb to six decimals.

### 3 Results

**Learning curves.** Pretraining lowers validation bpb from 3.4564 to 2.2882 in 300 steps (Figure 1). The base evaluation on 16,384 tokens per split gives 2.2947 bpb on training data and 2.3622 on validation data. SFT then lowers validation bpb from 2.2892 to 2.1456 in 40 steps. Asked “What is the capital of France?”, the SFT model replies with nonsense (`ATedning the date that is - min B`, whitespace collapsed), as expected at 0.84M parameters; the demo’s 73.5M model answered “Paris”.

**Probes.** Both probes repeat the 300-step schedule with one change (Table 2). Doubling the Muon matrix learning rate is neutral through step 100 and then slightly better, ending 0.012 bpb lower. The 2,048-token vocabulary is worse at every step; the gap narrows from 0.23 to 0.10 bpb but does not close. The demo’s own probe results are not part of the copied evidence, so whether these directions match the demo’s is not checked.

**Cost.** On a first run with an empty data cache, the whole pipeline takes 2 min 57 s, of which the baseline stages take 1 min 29 s. Pretraining takes 42.3 s, SFT 20.5 s, and the two probes 47.1 s and 38.4 s. The largest process peaks at 1.0 GB of RAM (the data download); training stays at or below 0.94 GB. The run downloads about 0.20 GB of data and uses about 0.9 GB of disk inside the project, mostly for the Python environment.

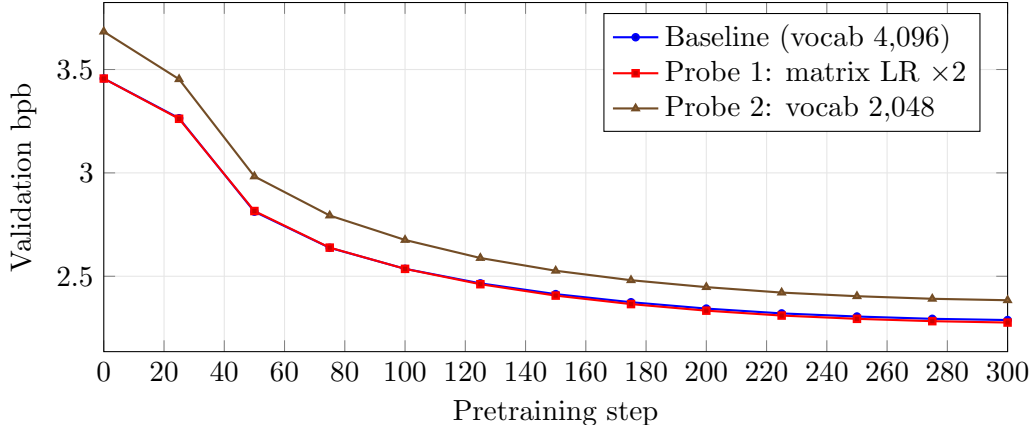


Figure 1: Validation bpb during pretraining for the baseline and both probes, drawn from `Code/results/training_metrics.csv`.

| Step | Baseline | Probe 1 (LR $\times 2$ ) | $\Delta$ | Probe 2 (vocab 2,048) | $\Delta$ |
|------|----------|--------------------------|----------|-----------------------|----------|
| 0    | 3.4564   | 3.4564                   | 0.0000   | 3.6833                | +0.2269  |
| 50   | 2.8124   | 2.8155                   | +0.0031  | 2.9829                | +0.1704  |
| 100  | 2.5360   | 2.5358                   | -0.0002  | 2.6756                | +0.1396  |
| 200  | 2.3436   | 2.3337                   | -0.0100  | 2.4474                | +0.1038  |
| 300  | 2.2882   | 2.2761                   | -0.0121  | 2.3840                | +0.0958  |

Table 2: Validation bpb of the probes against the baseline;  $\Delta$  = probe - baseline, lower is better.

## 4 Limitations

This is a demonstration of the pipeline, not a model: the chat replies are nonsense, and the bpb values and probe deltas say nothing about larger models. Each probe uses one seed; runs are deterministic, so the deltas are not run-to-run noise, but seed-to-seed variance was not measured, and probe 1’s  $-0.012$  bpb may not be meaningful. Probe 2 changes more than the vocabulary: the smaller model has 0.44M instead of 0.84M parameters and sees fewer bytes of text per 4,096-token step. The automatic weight-decay scaling gives  $\lambda = 7.0$  at this size, far outside the range the scaling rules were tuned for. SFT sees mostly short conversations, because the loader skips those longer than the 256-token context. Only this Windows machine was tested.

## 5 Reproducibility

From the project’s `Code/nanochat/` directory, `bash runs/runcpu_small.sh` all runs every stage and writes `Code/results/summary.json` and `Code/results/training_metrics.csv`; the timings above come from the first run recorded in `Code/README.md`. This paper builds from `paper/` with `latexmk -pdf main.tex` and draws Figure 1 from the tracked CSV at build time.

## References

- [1] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivas-

tav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Cl  mentine Fourier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. SmolLM2: When smol goes big – data-centric training of a small language model. arXiv:2502.02737, 2025.

- [2] alphaXiv. OpenResearch: Turn your coding agents into research agents. <https://github.com/alphaXiv/OpenResearch>, 2026. GitHub repository; demo `demo/nanochat` at commit 27cb342.
- [3] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. arXiv:2110.14168, 2021.
- [4] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. arXiv:2009.03300.
- [5] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks. <https://kellerjordan.github.io/posts/muon/>, 2024.
- [6] Andrej Karpathy. nanochat: The best ChatGPT that \$100 can buy. <https://github.com/karpathy/nanochat>, 2025. GitHub repository.
- [7] Andrej Karpathy. climbmix-400b-shuffle. <https://huggingface.co/datasets/karpathy/climbmix-400b-shuffle>, 2026. Hugging Face dataset.